The Curiously Recurring Template Pattern

CRTP is a particular form of a more general bounded polymorphism. While not a general replacement for **virtual functions**, it provides the C++ programmer with a similar tool that, under the right circumstances, offers several advantages.

What is wrong with a virtual function?

The problem is the performance overhead. A virtual function call can be several times more expensive than a non-virtual call, more for very simple functions that would have been inlined were they not virtual (a virtual function can never be inlined).

We can measure the overhead of a virtual function call. We are going to compare a very simple virtual function, with the minimum amount of code, against a non-virtual function doing the same thing. Here is our virtual function:

```
// Example 01
class B {
  public:
  B() : i_(0) {}
  virtual ~B() {}
  virtual void f(int i) = 0;
  int get() const { return i_; }
  protected:
  int i_;
};
class D : public B {
  public:
  void f(int i) override { i_ += i; }
};
```

And, here is the equivalent non-virtual one:

```
// Example 01
class A {
  public:
  A() : i_(0) {}
  void f(int i) { i_ += i; }
  int get() const { return i_; }
  protected:
```

```
  int i_;
};
```

We can now call both of them in a micro-benchmark fixture and measure how long each call takes:

```
void BM_none(benchmark::State& state) {
  A* a = new A;
  int i = 0;
  for (auto _ : state) a->f(++i);
  benchmark::DoNotOptimize(a->get());
  delete a;
}
void BM_dynamic(benchmark::State& state) {
  B* b = new D;
  int i = 0;
  for (auto _ : state) b->f(++i);
  benchmark::DoNotOptimize(b->get());
  delete b;
}
```

The **benchmark::DoNotOptimize** wrapper prevents the compiler from optimizing away the unused object, and, along with it, removing the entire set of function calls as unnecessary. Note that there is a subtlety in measuring the virtual function call time; a simpler way to write the code would be to avoid the **new** and **delete** operators and simply construct the derived object on the stack:

```
void BM_dynamic(benchmark::State& state) {
  D d;
  int i = 0;
  for (auto _ : state) d.f(++i);
  benchmark::DoNotOptimize(b->get());
}
```

However, this benchmark is likely to yield the same time as the non-virtual function call. The reason is not that a virtual function call has no overhead. Rather, in this code, the compiler is able to deduce that the call to the virtual function, **f()**, is always a call to **D::f()**. A decent optimizing compiler will de-virtualize such a call, for instance, generating a direct call to **D::f()** without the indirection and the reference to the *v-table*. Such a call can even be inlined.

Another possible complication is that both microbenchmarks, especially the non-virtual call, may be too fast: the body of the benchmark loop is likely to take less time than the overhead of the loop.

We can remedy that by making several calls inside the body of the loop. This can be accomplished with the **C++ preprocessor** macros:

```
#define REPEAT2(x) x x
#define REPEAT4(x) REPEAT2(x) REPEAT2(x)
#define REPEAT8(x) REPEAT4(x) REPEAT4(x)
#define REPEAT16(x) REPEAT8(x) REPEAT8(x)
#define REPEAT32(x) REPEAT16(x) REPEAT16(x)
#define REPEAT(x) REPEAT32(x)
```

Now, inside the benchmark loop, we can write the following:

```
REPEAT(b->f(++i);)
```

The per-iteration time reported by the benchmark now refers to **32** function calls. While this does not matter for comparing the two calls, it may be convenient to make the benchmark itself report the true number of calls per second by adding this line to the end of the benchmark fixture, after the loop:

```
state.SetItemsProcessed(32*state.iterations());
```

We can now compare the results of the two benchmarks:

```
Benchmark           Time UserCounters...
BM_none          1.60 ns items_per_second=19.9878G/s
BM_dynamic       9.04 ns items_per_second=3.54089G/s
```

We see that the virtual function call is almost 10 times more expensive than the non-virtual one.

Introducing CRTP

`CRTP` turns inheritance on its head:

```
template <typename D> class B {
  ...
};
class D : public B<D> {
```

```
  ...
};
```

The first change that jumps out is that the base class is now a **class** template. The derived class still inherits from the base class, but now from the specific instantiation of the base class template: on its own! Class **B** is instantiated on class **D**, and class **D** inherits from that instantiation of class **B**, which is instantiated on class **D**, which inherits from class **B**, which... that's recursion in action.

What is the motivation for this pattern? Consider that now the base class has compile-time information about the derived class. Therefore, what used to be a virtual function call can now be bound to the right function at compile time:

```
// Example 01
template <typename D> class B {
  public:
  B() : i_(0) {}
  void f(int i) { static_cast<D*>(this)->f(i); }
  int get() const { return i_; }
  protected:
  int i_;
};
class D : public B<D> {
  public:
  void f(int i) { i_ += i; }
};
```

The call itself can still be done on the base class pointer:

```
B<D>* b = ...;
b->f(5);
```

There is no indirection and no overhead for the virtual call. The compiler can, at compile time, track the call all the way to the actual function called, and even inline it:

```
void BM_static(benchmark::State& state) {
  B<D>* b = new D;
  int i = 0;
  for (auto _ : state) {
    REPEAT(b->f(++i);)
  }
  benchmark::DoNotOptimize(b->get());
```

```
  state.SetItemsProcessed(32*state.iterations());
}
```

The benchmark shows that the function call made through the `CRTP` takes exactly as much time as a regular function call:

```
Benchmark            Time UserCounters...
BM_none           1.60 ns items_per_second=19.9878G/s
BM_dynamic        9.04 ns items_per_second=3.54089G/s
BM_static         1.55 ns items_per_second=20.646G/s
```

The main restriction on the CRTP is that the size of the base class, **B**, cannot depend on its template parameter, **D**. More generally, the template for class **B** has to instantiate with type **D** being an incomplete type. For example, this will not compile:

```
template <typename D> class B {
  using T = typename D::T;
  T* p_;
};
class D : public B<D> {
  using T = int;
};
```

The realization that this code will not compile may come be a surprise, given how similar it is to many widely used templates that refer to the nested types of their template parameters. For example, consider this template, which converts any sequence container with **push_back()** and **pop_back()** functions to a stack:

```
template <typename C> class stack {
  C c_;
  public:
  using value_type = typename C::value_type;
  void push(const valuetype& v) { c.push_back(v); }
  value_type pop() {
    value_type v = c.back();
    c.pop_back();
    return v;
  }
};
stack<std::vector<int>> s;
```

Note that the **using** type alias for **value_type** looks exactly the same as the preceding one, in our attempt to declare class **B**. So, what is wrong with the one in **B**? Actually, nothing is wrong with class **B** itself. It would compile just fine in a context similar to that of stack class:

```
class A {
  public:
  using T = int;
  T x_;
};
B<A> b; // Compiles with no problems
```

The problem lies not with class **B** itself, but with our intended use of it:

```
class D : public B<D> ...
```

At the point where **B** has to be known, type **D** has not been declared yet. It cannot be: the declaration of class **D** requires us to know exactly what the base class `B<D>` is. So, if class **D** has not been declared yet, how does the compiler know that the identified **D** even refers to a type? After all, you cannot instantiate a template on a completely unknown type. The answer lies somewhere in between: class **D** is forward-declared, the same as if we had this code:

```
class A;
B<A> b; // Now does not compile
```

Some templates can be instantiated on forward-declared types, while others cannot. The gist is : anything that might affect the size of the class has to be fully declared. A reference to a type declared inside an incomplete type, such as our **using T = typename D::T**, would be a forward declaration of a nested class, and those are not allowed either.

On the other hand, the body of a member function of a class template is not instantiated until it's called. In fact, for a given template parameter, a member function does not even have to compile, as long as it's not called. Therefore, references to the derived class, its nested types, and its member functions, inside the **member functions of the base class** are perfectly fine. Also, since the derived class type is considered forward-declared inside the base class, we can declare pointers and references to it. Here is a very common refactoring of the `CRTP` base class that consolidates the uses of the `static_cast` in one place:

```
template <typename D> class B {
  ...
  void f(int i) { derived()->f(i); }
```

```
  D* derived() { return static_cast<D*>(this); }
};
class D : public B<D> {
  ...
  void f(int i) { i_ += i; }
};
```

The base class declaration owns a pointer to the incomplete (forward-declared) type **D**. It works like any other pointer to an incomplete type; by the time the pointer is de-referenced, the type has to be complete. In our case, this happens inside the body of the member function; **B::f()**, which, as we discussed, is not compiled until it's called by the client code.

So what do we do if we need to use a nested type of the derived class while writing the base class? Inside a function body, there is no problem. If we need to use the nested type in the base class itself, it is usually for one of two reasons. The first is to **declare the return type of a member function**:

```
// Example 01a
template <typename D> class B {
  typename D::value_type get() const {
    return static_cast<const D*>(this)->get();
  }
  …
};
D : public B<D> {
  using value_type = int;
  value_type get() const { … };
  …
};
```

This will not compile. Fortunately, this problem is easy to solve, all we have to do is let the compiler deduce the return type:

```
// Example 01a
template <typename D> class B {
  auto get() const {
    return static_cast<const D*>(this)->get();
  }
  …
};
```

The second case is more difficult: **the nested type is needed to declare a data member or a parameter**. In this case, there is only one option left: **the type should be passed into the base class as an additional template parameter**, but it introduces some redundancy into the code:

```
// Example 01a
template <typename T, typename value_type> class B {
  value_type value_;
  …
};
class D : public B<D, int> {
  using value_type = int;
  value_type get() const { … }
  …
};
```

CRTP and static polymorphism

`CRTP` can be used to allow the derived class to customize the behavior of the base class:

```
template <typename D> class B {
  public:
  ...
  void f(int i) { static_cast<D*>(this)->f(i); }
  protected:
  int i_;
};
class D : public B<D> {
  public:
  void f(int i) { i_ += i; }
};
```

If the base class **B::f()** method is called, it dispatches the call to the derived class method for the real derived class, just like a virtual function does. In order to fully take advantage of this polymorphism, we have to be able to call the methods of the base class through the base class pointer. Without this ability, we are simply calling methods of the derived class whose type we already know:

```
D* d = ...; // Get an object of type D
d->f(5);
B<D>* b = ...; // Also has to be an object of type D
b->f(5);
```

The function call looks exactly like any virtual function class, with the base class pointer. The actual function, **f()**, that is called comes from the derived class, **D::f()**. There is a significant difference: the actual type of the derived class, **D**, has to be known at compile time.

The base class pointer is not **B*** but rather **B***, which implies that the derived object is of type **D**. There does not seem to be much point to such *polymorphism* if the programmer has to know the actual type. But, that is because we have not fully thought through what *compile-time polymorphism* really means. Just as the benefit of a virtual function is that we can call member functions of a type we don't even know exists, the same has to be true for *static polymorphism* to be useful.

How do we write a function that has to compile for parameters of an unknown type? With a function template, of course:

```
// Example 01
template <typename D> void apply(B<D>* b, int& i) {
  b->f(++i);
}
```

This is a template function that can be called on any base class pointer, and it automatically deduces the type of the derived class, **D**. Now, we can write what looks like regular polymorphic code:

```
B<D>* b = new D;    // 1
apply(b);          // 2
```

Note that, on line one, the object has to be constructed with the knowledge of the actual type. This is always the case; the same is true for regular runtime polymorphism with virtual functions:

```
void apply(B* b) { ... }
B* b = new D;    // 1
apply(b);        // 2
```

In both cases, on line two, we invoke some code that was written only with the knowledge of the base class. The difference is that, with **run-time polymorphism**, we have a **common base class** and some functions that operate on it. With CRTP and static polymorphism, there is a **common base class template** but **no single common base class** (a template is not a type) and every function that operates on this base template becomes a template itself.

The compile-time pure virtual function

A pure virtual function must be implemented in all derived classes. A class that declares a pure virtual function, or inherits one and does not override it, is an abstract class; it can be further derived from, but it cannot be instantiated.

When we contemplate the equivalent of a pure virtual function for static polymorphism, we realize that our CRTP implementation suffers from a major vulnerability. What happens if we forget to override the *compile-time virtual function,* **f()**, in one of the derived classes?

```
// Example 02
template <typename D> class B {
  public:
  ...
  void f(int i) { static_cast<D*>(this)->f(i); }
};
class D : public B<D> {
  // no f() here!
};
...
B<D>* b = ...;
b->f(5); // 1
```

This code compiles with no errors or warnings: on line `1`, we call **B::f()**, which, in turn, calls **D::f()**. Class **D** does not declare its own version of the member **f()**, so the one inherited from the base class is the one that is called. That is, of course, the member function, **B::f()**, that we have already seen, which again calls **D::f()**, which is really **B::f()** ... and we have an infinite loop.

The problem here is that nothing requires us to override the member function **f()** in the derived class, but the program is malformed if we don't. The root of the problem is that we are mixing together the interface and the implementation: the public member function declaration in the base class says that all derived classes must have a function, **void f(int)**, as a part of their public interface. The derived class's version of the same function provides the actual implementation. It would be a lot easier if these functions had different names:

```
// Example 03
template <typename D> class B {
  public:
  ...
  void f(int i) { static_cast<D*>(this)->f_impl(i); }
};
class D : public B<D> {
```

```
  void f_impl(int i) { i_ += i; }
};
...
B<D>* b = ...;
b->f(5);
```

What happens now if we forget to implement **D::f_impl()**? The code does not compile, because there is no such member function in class **D**, either directly or through inheritance. So, we have implemented a compile-time pure virtual function! Note that the virtual function is actually **f_impl()**, not **f()**.

With that accomplished, how would we implement a regular virtual function, with a default implementation that can be optionally overridden? If we follow the same pattern of separating the interface and the implementation, we only have to provide the default implementation for **B::f_impl()**:

```
// Example 03
template <typename D> class B {
  public:
  ...
  void f(int i) { static_cast<D*>(this)->f_impl(i); }
  void f_impl(int i) {}
};
class D1 : public B<D1> {
  void f_impl(int i) { i_ += i; }
};
class D2 : public B<D2> {
  // No f() here
};
...
B<D1>* b = ...;
b->f(5); // Calls D1::f_impl()
B<D2>* b1 = ...;
b1->f(5); // Calls B::f_impl() by default
```

Destructors and polymorphic deletion

In many cases, polymorphic deletion is not a concern. All objects are created with their actual types known. If the code that constructs the objects also owns and eventually deletes them, the question *what is the type of the deleted object?* is never really asked. Similarly, if the objects are stored in a container, they are not deleted through the base class pointer or reference:

```
template <typename D> void apply(B<D>& b) {
  ... operate on b ...
}
{
  std::vector<D> v;
  v.push_back(D(...)); // Objects created as D
  ...
  apply(v[0]); // Objects processed as B&
} // Objects deleted as D
```

In many cases, the objects are constructed and deleted with their actual type known and no polymorphism involved, but the code that works on them in between is universal, written to work on the base type and, therefore, any class derived from that base type.

But, what if we need to actually delete the object through the base class pointer?. First of all, a simple call to the **delete** operator will do the wrong thing:

```
B<D>* b = new D;
...
delete b;
```

This code compiles. Even the compilers that normally warn when a class has a virtual function but a non-virtual destructor **do not generate any warnings** in this case, because there are no virtual functions, and CRTP polymorphism is not recognized by the compiler as a potential source of trouble. However, the trouble there is that only the destructor of the base class, **B**, itself is called; the destructor of **D** is never called!

It is tempted to resolve this problem by casting to the known derived type and calling the indented member function of the derived class:

```
template <typename D> class B {
  public:
  ~B() { static_cast<D*>(this)->~D(); }
};
```

Unlike the regular functions, this attempt is badly broken for two reasons

1. In the destructor of the base class, the actual object is not of the derived type anymore, and calling any member functions of the derived class on it results in undefined behavior.

2. Even if this worked, the destructor of the derived class is going to do its work and then call the destructor of the base class, which results in an infinite loop.

There are two solutions to this problem. One option is to extend the compile-time polymorphism to the act of deletion in the same way as any other operation, with a function template:

```
template <typename D> void destroy(B<D>* b) {
  delete static_cast<D*>(b);
}
```

This is well-defined. The **delete** operator is called on the pointer of the actual type, **D**, and the right destructor is called. However, these objects must be deleted using this **destroy()** function, instead of calling the **delete** operator.

The second option is to actually make the destructor *virtual*. This does bring back the overhead of the virtual function call, but only for the destructor. It also increases the object size by the size of the virtual pointer.

CRTP and access control

When implementing CRTP classes, you do have to worry about access: any method you want to call has to be accessible. Either the method has to be public, or the caller has to have special access. This is a little different from the way virtual functions are called: when calling a virtual function, the caller must have access to the member function that is named in the call. For example, a call to the base class function, **B::f()**, requires that either **B::f()** is public or the caller has access to non-public member functions (another member function of class **B** can call **B::f()** even if it's private). Then, if **B::f()** is virtual and overridden by the derived class **D**, the override **D::f()** is actually called at **runtime**. There is no requirement that **D::f()** be accessible from the original call site; for example, **D::f()** can be private.

The situation with CRTP polymorphic calls is somewhat different. All calls are explicit in the code, and the callers must have access to the functions they call. Usually, it means that the base class must have access to the member functions of the derived class. Consider the following example with explicit access control:

```
template <typename D> class B {
  public:
  ...
  void f(int i) { static_cast<D*>(this)->f_impl(i); }
  private:
  void f_impl(int i) {}
};
```

```
class D : public B<D> {
  private:
  void f_impl(int i) { i_ += i; }
  friend class B<D>;
};
```

Here, both functions, **B::f_impl()** and **D::f_impl()**, are private in their respective classes. The base class has no special access to the derived class, and cannot call its private member functions. Unless we want to change the member function, **D::f_impl()**, from private to public and allow any caller access to it, we have to declare the base class to be a **friend** of the derived class.

There is also some benefit in doing the reverse. Suppose a new derived class, **D1**, with a different override of the implementation function, **f_impl()**:

```
class D1 : public B<D> {
  private:
  void f_impl(int i) { i_ -= i; }
  friend class B<D1>;
};
```

This class has a subtle error: it is not actually derived from **B** but from the old class, **B**; a mistake that is easy to make when creating a new class from an old template. This mistake will be found if we attempt to use the class polymorphically:

```
B<D1>* b = new D1;
```

This does not compile because **B** is not a base class of **D1**. However, not all uses of CRTP involve a polymorphic call. In any case, it would be better if the error was caught when class **D1** was first declared. We can accomplish that by making class **B** into a sort of abstract class, only in the sense of **static polymorphism**. All it takes is to make the constructor of class **B** private and to **declare the derived class to be a friend**:

```
template <typename D> class B {
  int i_;
  B() : i_(0) {}
  friend D;
  public:
  void f(int i) { static_cast<D*>(this)->f_impl(i); }
  private:
```

```
  void f_impl(int i) {}
};
```

Note the somewhat unusual form of the friend declaration: **friend D** and not **friend class D**. This is how to write a friend declaration for the template parameter. Now, the only type that can construct an instance of class **B** is that specific derived class, **D**, which is used as the template parameter, and the erroneous code, **class D1 : public B**, does not compile any longer.

CRTP as a delegation pattern

In CRTP, the interface and the generic code are in the base class, while the derived class overrides the specific implementation. This relation continues to hold when a CRTP object is accessed through the base class pointer or reference. Such use of CRTP is also called a **static interface**.

When the derived object is used directly, the situation is quite different: the base class is no longer the interface, and the derived class is not just the implementation. The derived class **expands the interface of the base class**, and the base class delegates some of its behavior to the derived class.

Expanding the interface

There are several examples where CRTP is used to delegate behavior from the base class to the derived one.

The first example is a very simple one: for any class that provides **operator+=()**, we want to generate **operator+()** automatically that used the former:

```
// Example 04
template <typename D> struct plus_base {
  D operator+(const D& rhs) const {
    D tmp = rhs;
    tmp += static_cast<const D&>(*this);
    return tmp;
  }
};
class D : public plus_base<D> {
  int i_;
  public:
  explicit D(int i) : i_(i) {}
  D& operator+=(const D& rhs) {
    i_ += rhs.i_;
    return *this;
```

```
  }
};
```

Any class that inherits from **plus_base** automatically acquires the addition operator that is guaranteed to match the provided increment operator. But how the `operator+` declared here is strange.

Aren't two-argument operators supposed to be non-member functions? Indeed, they usually are. Nothing in the standard requires them to be, and the preceding code is technically valid. The reason binary operators such as ==, +, and so on are usually declared as non-member functions has to do with implicit conversions: if we have an addition, **x** + **y**, and the intended **operator+** is a member function, it has to be a member function of the **x** object. Not any object implicitly convertible to the type of **x**, except **x** itself: it's a member function call on **x**.

In contrast, the **y** object has to be implicitly convertible to the type of the argument of that member function **operator+**, i.e. **x**. To restore the symmetry and allow implicit conversions (if any are provided) on both the left- and right-hand side of the + sign, we have to declare **operator+** as a non-member function. Usually, such a function needs to have access to the private data members of the class, so it has to be declared a **friend**. Putting all of this together, we arrive at this alternative implementation:

```
// Example 05
template <typename D> struct plus_base {
  friend D operator+(const D& lhs, const D& rhs) {
    D tmp = lhs;
    tmp += rhs;
    return tmp;
  }
};
class D : public plus_base<D> {
  int i_;
  public:
  explicit D(int i) : i_(i) {}
  D& operator+=(const D& rhs) {
    i_ += rhs.i_;
    return *this;
  }
};
```

There is a significant difference between this use of CRTP and the one we saw earlier: the object that is going to be used in the program is of type **C**, and it will never be accessed through a pointer to **plus_base**. The latter isn't a complete interface for anything but is really an implementation that

makes use of the interface provided by the derived class. **CRTP here is used as an implementation technique**, not as a design pattern.

One example is the generated comparison and ordering operations. In C++20, the recommended choice for designing interfaces of value types (or any other comparable and ordered types) is to provide only two operators, **operator==()** and **operator<=>()**. The compiler will generate the rest. If you like this approach to interface design and want to use it in earlier versions of C++, you need a way to implement it. CRTP offers us a possible implementation. We will need a base class that will generate **operator!=()** from the **operator==()** of the derived class. It will also generate all ordering operators; of course, we cannot use **operator<=>()** before C++20, but we can use any member function name we agree on, such as **cmp()**:

```
template <typename D> struct compare_base {
  friend bool operator!=(const D& lhs, const D& rhs) {
    return !(lhs == rhs); }
  friend bool operator<=(const D& lhs, const D& rhs) {
    return lhs.cmp(rhs) <= 0;
  }
  friend bool operator>=(const D& lhs, const D& rhs) {
    return lhs.cmp(rhs) >= 0;
  }
  friend bool operator< (const D& lhs, const D& rhs) {
    return lhs.cmp(rhs) <  0;
  }
  friend bool operator> (const D& lhs, const D& rhs) {
    return lhs.cmp(rhs) >  0;
  }
};
class D : public compare_base<D> {
  int i_;
  public:
  explicit D(int i) : i_(i) {}
  auto cmp(const D& rhs) const {
    return (i_ < rhs.i_) ? -1 : ((i_ > rhs.i_) ? 1 : 0);
  }
  bool operator==(const D& rhs) const {
    return i_ == rhs.i_;
  }
};
```

CRTP and concepts

Concepts are all about restricting interfaces, while CRTP extends the interfaces. There are discussions inspired by cases where both concepts and CRTP can solve the same problem by totally different means.

Recall the example of CRTP to automatically generate **operator+()** from **operator+=()**; all we had to do was to inherit from the special base class template:

```
// Example 05
template <typename D> struct plus_base {
  friend D operator+(const D& lhs, const D& rhs) { … }
};
class D : public plus_base<D> {
  D& operator+=(const D& rhs) { … }
};
```

Our base class serves two purposes. *First*, it generates **operator+()** from **operator+=()**. *Second*, it provides a mechanism for classes to opt into this automation: to receive the generated **operator+()**, a class must inherit from **plus_base**.

The first problem by itself is easy to solve, we can just define a global **operator+()** template:

```
template <typename T>
T operator+(const T& lhs, const T& rhs) {
  T tmp = lhs;
  tmp += rhs;
  return tmp;
}
```

There is a problem with this template: we just provided a global **operator+()** for every type in our program, whether it needs one or not. Furthermore, most of the time it won't even compile since not all classes define **operator+=()**.

This is where the concepts come in: we can restrict the applicability of our new **operator+()** so, in the end, it is generated for the same types that we would have otherwise inherited from **plus_base** and no other.

One way to do this is to require that the template parameter type `T` at least have the increment operator:

```
template <typename T>
requires( requires(T a, T b) { a += b; } )
T operator+(const T& lhs, const T& rhs) { … }
```

However, this is not the same result as what we got with CRTP. In some cases, it may be a better result: instead of having to opt every class into the automatic generation of **operator+()**, we did it for every class that satisfies certain restrictions.

But in other cases, any reasonable description of these restrictions produces overly broad results, and we have to opt into our types one by one. This is easy to do with concepts as well, but the technique used is not widely known. All you need to do is to define a concept whose general case is `false` (a **Boolean** variable will suffice):

```
template <typename T>
constexpr inline bool gen_plus = false; // General
template <typename T>
requires gen_plus<T>
T operator+(const T& lhs, const T& rhs) { … }
```

Then, for every type that needs to opt in, we specialize the concept:

```
class D { // No special base
  D& operator+=(const D& rhs) { … }
};
template <>
constexpr inline bool generate_plus<D> = true; // Opt-in
```

There are some advantages to both approaches: CRTP uses a base class that can be more complex than just a wrapper for a definition of an operator; while concepts can combine explicit opt-in with some more general restrictions when appropriate.

However, these discussions miss a much more important distinction: CRTP can be used to expand class interface with both member and non-member functions, while concepts can be used only with non-member functions, including non-member operators.

Of course, we can use concepts with CRTP instead of trying to replace CRTP: if the CRTP base class template has some requirements on the type we want to derive from it, we can enforce these with concepts.

CRTP as an implementation technique

CRTP is often used as a purely implementation pattern but it can influence design: some design choices are desirable but hard to implement, and, if a good implementation technique comes along, the design choices often change.

CRTP for code reuse

A specific implementation issue is that multiple classes have some common code. Ordinarily, we would write a base class for them. But the common code is not really common: it does the same thing for all classes except it for the types it uses. What we need is not a common base class but a common base class template. And that brings CRTP.

An example is an object registry. It may be desirable to know how many objects of a certain type are currently in existence, and perhaps to maintain a list of such objects. We definitely do not want to instrument every class with the registry mechanism, so we want to move it to the base class. But, now we have a problem: if we have two derived classes, **C** and **D**, both inherit from the same base class, **B**, and the count of instances of **B** will be the total for both **C** and **D**.

The problem is not that the base class can't determine what the real type of the derived class is: it can, if we're willing to pay the cost of runtime polymorphism. The problem is that the base class has only one counter (or however many are coded in the class), while the number of different derived classes is unlimited. We could implement a very complex, expensive, and non-portable solution that uses **Run-Time Type Information** (**RTTI**), such as **typeid**, to determine the class name and maintain a map of names and counters. But, what we really need is one counter per derived type, and the only way to do it is to make the base class aware of the derived class type at compile time. This is what CRTP can do:

```
// Example 08
template <typename D> class registry {
  public:
  static size_t count;
  static D* head;
  D* prev;
  D* next;
  protected:
  registry() {
    ++count;
    prev = nullptr;
    next = head;
    head = static_cast<D*>(this);
    if (next) next->prev = head;
  }
```

```
  registry(const registry&) {
    ++count;
    prev = nullptr;
    next = head;
    head = static_cast<D*>(this);
    if (next) next->prev = head;
  }
  ~registry() {
    --count;
    if (prev) prev->next = next;
    if (next) next->prev = prev;
    if (head == this) head = next;
  }
};
template <typename D> size_t registry<D>::count(0);
template <typename D> D* registry<D>::head(nullptr);
```

We have declared the constructor and the destructor protected because we don't want any registry objects created, except by the derived classes. It is also important to not forget the copy constructor, otherwise, the default one is generated by the compiler, and it does not increment the counter or update the list (but the destructor does, so the counter will go negative and overflow).

For each derived class, **D**, the base class is **registry**, which is a separate type with its own static data members, **count** and **head** (the latter is the head of the list of currently active objects). Any type that needs to maintain the runtime registry of active objects now only needs to inherit from **registry**:

```
// Example 08
class C : public registry<C> {
  int i_;
  public:
  C(int i) : i_(i) {}
};
```

CRTP for generic interfaces

Another scenario where it is necessary to delegate behavior to derived classes is the problem of visitation. **Visitors**, in a general sense, are objects that are invoked to process a collection of data objects and execute a function on each one in turn.

In most scenarios, there are hierarchies of visitors, where the derived classes customize or alter some aspect of the behavior of the base classes. While the most common implementation of

visitors uses dynamic polymorphism and virtual function calls, a static visitor offers the same sort of performance benefits.

Visitors are not usually invoked polymorphically; you create the visitor you want and run it. The base visitor class call the member functions that may be dispatched, at compile time, to the derived classes if they have the right overrides. Consider this generic visitor for a collection of animals:

```
// Example 09
struct Animal {
  public:
  enum Type { CAT, DOG, RAT };
  Animal(Type t, const char* n) : type(t), name(n) {}
  const Type type;
  const char* const name;
};
template <typename D>
class GenericVisitor {
  public:
  template <typename it> void visit(it from, it to) {
    for (it i = from; i != to; ++i) {
      this->visit(*i);
    }
  }
  private:
  D& derived() { return *static_cast<D*>(this); }
  void visit(const Animal& animal) {
    switch (animal.type) {
      case Animal::CAT:
        derived().visit_cat(animal); break;
      case Animal::DOG:
        derived().visit_dog(animal); break;
      case Animal::RAT:
        derived().visit_rat(animal); break;
    }
  }
  void visit_cat(const Animal& animal) {
    cout << "Feed the cat " << animal.name << endl;
  }
  void visit_dog(const Animal& animal) {
    cout << "Wash the dog " << animal.name << endl;
  }
  void visit_rat(const Animal& animal) {
  cout << "Eeek!" << endl;
}
  friend D;
```

```
  GenericVisitor() = default;
};
```

Note that the main visitation method is a template member function (a template within a template!), and it accepts any kind of iterator that can iterate over a sequence of **Animal** objects.

Also, by declaring a private default constructor at the bottom of the class, we are protecting ourselves from making a mistake where the derived class incorrectly specifies its own type for the inheritance. Now, we can start creating some visitors. The default visitor simply accepts the default actions provided by the generic visitor:

```
class DefaultVisitor :
  public GenericVisitor<DefaultVisitor> {
};
```

We can visit any sequence of **Animal** objects, for example, a `vector`:

```
std::vector<Animal> animals {
  {Animal::CAT, "Fluffy"},
  {Animal::DOG, "Fido"},
  {Animal::RAT, "Stinky"}};
DefaultVisitor().visit(animals.begin(), animals.end());
```

But, we don't have to constrain ourselves to the default actions: we can override the visitation actions for one or more animal types:

```
class TrainerVisitor :
  public GenericVisitor<TrainerVisitor> {
  friend class GenericVisitor<TrainerVisitor>;
  void visit_dog(const Animal& animal) {
    cout << "Train the dog " << animal.name << endl;
  }
};
class FelineVisitor :
  public GenericVisitor<FelineVisitor> {
  friend class GenericVisitor<FelineVisitor>;
  void visit_cat(const Animal& animal) {
    cout << "Hiss at the cat " << animal.name << endl;
  }
  void visit_dog(const Animal& animal) {
    cout << "Growl at the dog " << animal.name << endl;
```

```
  }
  void visit_rat(const Animal& animal) {
    cout << "Eat the rat " << animal.name << endl;
  }
};
```

CRTP and policy-based design

CRTP can provide additional functionality to the derived classes. If we want to extend the interface of the derived class in multiple ways, a single base class presents an unnecessary restriction.

First of all, if we add several member functions, the base class can get quite large. *Second*, we may want a more modular approach to the interface design. For example, we can have one base class template that adds factory construction methods to any derived class:

```
// Example 10
template <typename D> struct Factory {
  template <typename... Args>
  static D* create(Args&&... args) {
    return new D(std::forward<Args>(args)...);
  }
  static void destroy(D* d) { delete d; }
};
```

We can even have several different factories that present the same interface but allocate memory differently. We can have another base class template that adds the conversion to string to any class that has the stream inserter operator:

```
// Example 10
template <typename D> struct Stringify {
  operator std::string() const {
    std::stringstream S;
    S << *static_cast<const D*>(this);
    return S.str();
  }
};
```

It makes no sense to combine the two into a single base. In a large system, there can be many more of these classes, each adds a specific feature to the derived class and uses CRTP to implement it.

But not every derived class needs every one of these features. With multiple base classes to choose from, it is easy to construct a derived class that has a specific set of features:

```
// Example 10
class C1 : public Stringify<C1>, public Factory<C1> {…};
```

This works, but we are in danger of repeating nearly identical code if we need to implement several derived classes that have very similar behavior, except for the features provided by the CRTP bases.

For example, if we have another factory that constructs objects in thread-local memory to speed up the performance of concurrent programs (let's call it **TLFactory**), we might have to write this:

```
class C2 : public Stringify<C2>, public TLFactory<C2> {…};
```

But the two classes **C1** and **C2** are exactly the same except for the base classes, and we would have to implement and maintain two copies of identical code.

It would be better if we could write a single class template and plug different base classes into it as needed. This is the main idea of policy-based design. Since we need a class template that accepts multiple base class types, we will need to use a variadic template. We need something like this:

```
template <typename… Policies>
class C : public Policies… {};
```

But the above code won't compile if we try to use **Factory** or **Stringify** as policies. The reason is that they are not types (classes) and, therefore, cannot be used as a type name. They are templates, so we have to declare the template parameters of the template **C** as templates themselves (this is known as a template template parameter).

```
template <template <typename> class B> class C;
```

If we wanted to inherit from a specific instantiation of this class template **B**, we would write

```
template <template <typename> class B>
class C : public B<template argument> {…};
```

When using CRTP, the template argument is the type of the derived class itself, **C**:

```
template <template <typename> class B>
class C : public B<C<B>> {…};
```

Generalizing this to a parameter pack :

```
// Example 11
template <template <typename> class... Policies>
class C : public Policies<C<Policies...>>... {…};
```

The template parameter is a pack (any number of templates instead of a single class). The derived class inherits from the entire pack Policies..., except Policies are templates and we need to specify the actual instantiations of these templates.

Each template in the pack is instantiated on the derived class, whose type is C<Policies...>.

If we need additional template parameters, for example, to enable using different value types in the class C, we can combine them with policies:

```
// Example 11
template <typename T,
          template <typename> class... Policies>
class C : public Policies<C<T, Policies...>>... {
  T t_;
  public:
  explicit C(T t) : t_(t) {}
  const T& get() const { return t_; }
  friend std::ostream&
  operator<<(std::ostream& out, const C c) {
    out << c.t_;
    return out;
  }
};
```

To use this class with a particular set of policies, it is convenient to define some aliases:

```
using X = C<int, Factory, Stringify>;
```

If we want to use several classes with the same Policies, we can define a template alias as well:

```
template <typename T>
using Y = C<T, Factory, Stringify>;
```

Summary

The Curiously Recurring Template Pattern creates a circular loop, where the derived class inherits the interface and the implementation from the base class, while the base class has access to the interface of the derived class through the template parameters.

CRTP has two main use modes: true static polymorphism, or *static interface*, where the object is primarily accessed as the base type, and expanding the interface, or delegation, where the derived class is accessed directly but the implementation uses CRTP to provide common functionality. The latter can vary from a simple addition of one or two methods to a complex task of composing the interfaces of derived classes from multiple building blocks or policies.

Questions

1. How expensive is a virtual function call, and why?

   A: While not very expensive in absolute numbers (a few nanoseconds at most), a virtual function call is several times more expensive than a non-virtual one, and could easily be an order of magnitude or more slower than an inlined function call. The overhead comes from the indirection: a virtual function is always invoked by a function pointer, and the actual function is unknown at compile time and cannot be inlined.

2. Why does a similar function call, resolved at compile time, have no such performance overhead?

   A: If the compiler knows the exact function that is going to be called, it can optimize away the indirection and may be able to inline the function.

3. How would you make compile-time polymorphic function calls?

   A: Just like the runtime polymorphic calls are made through the pointer to the base class, the static polymorphic calls must be also made through a pointer or reference to the base class. In the case of CRTP and static polymorphism, the base type is actually a whole collection of types generated by the base class template, one for each derived class. To make a polymorphic call we have to use a function template that can be instantiated on any of these base types.

4. How would you use CRTP to expand the interface of the base class?

A: When the derived class is called directly, the use of CRTP is quite different from the compile-time equivalent of the virtual functions. It becomes an implementation technique, where common functionality is provided to multiple derived classes, and each one expands and customizes the interface of the base class template.

5. What is necessary to use multiple CRTP base classes in a single derived class?.

   A: Strictly speaking, nothing new is needed to use multiple CRTP bases: the derived class can inherit from several such base types, each an instantiation of a CRTP base class template. However, listing these based together with the correct template parameter (derived class itself) for each one becomes cumbersome. It is easier and less error-prone to declare the derived class as a variadic template with a template template parameter and inherit from the entire parameter pack.